1инистерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Кафедра информатики и вычислительной техники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины «ОБЪЕКТНО-ОРИЕНТИРОВАННОЕ ПРОГРАММИРОВАНИЕ»
Вариант №9

Выполнила:

Ковжого Е. А.

3 курс, группа ИВТ-б-о-24-1,

09.03.01 «Информатика и вычислительная техника», направленность (профиль) «Программное обеспечение средств вычислительной техники и автоматизированных систем», очная форма обучения

_____

(подпись)

Руководитель практики:
Воронкин Р. А., доцент департамента цифровых, робототехнических систем и электроники института перспективной инженерии.

_____

(подпись)

Отчет защищен с оценкой _____Дата защиты_____

Ставрополь, 2026 г.

Тема: перегрузка операторов в языке Python

Цель: приобретение навыков по перегрузке операторов при написании программ с помощью языка программирования Python версии 3.x

Порядок выполнения работы:

1. Создали репозиторий и выполнили клонирование репозитория.
2. Проработали примеры представленные в лабораторной работе.

Листинг кода pr1.py:

```
class A:
    def go(self):
        print("Go, A!")
class B(A):
    def go(self, name):
        print('Go, {}!'.format(name))
```

Листинг кода pr2.py:

```
class A:
    def __init__(self, name):
        self.name = name

a = A('Vasya')
print(a.name)
```

Листинг кода pr3.py:

```
import math



class Vector2D:
    def __init__(self, x, y):
```

```
    self.x = x
    self.y = y

def __repr__(self):
    return 'Vector2D({}, {})'.format(self.x, self.y)

def __str__(self):
    return '({}, {})'.format(self.x, self.y)

def __add__(self, other):
    return Vector2D(self.x + other.x, self.y + other.y)

def __iadd__(self, other):
    self.x += other.x
    self.y += other.y
    return self

def __sub__(self, other):
    return Vector2D(self.x - other.x, self.y - other.y)

def __isub__(self, other):
    self.x -= other.x
    self.y -= other.y
    return self

def __abs__(self):
    return math.hypot(self.x, self.y)

def __bool__(self):
```

```
        return self.x != 0 or self.y != 0

    def __neg__(self):
        return Vector2D(-self.x, -self.y)

x = Vector2D(3,4)
print(x)
print(abs(x))
y = Vector2D(5,6)
print(x+y)
print(x-y)
print(-x)
x+=y
print(x)
print(bool(x))
z = Vector2D(0,0)
print(bool(z))
print(-z)
```

Листинг кода pr4.py:

```
class Rational:

    def __init__(self, a=0, b=1):
        a = int(a)
        b = int(b)

        if b == 0:
            raise ValueError("Illegal value of the denominator")

        self.__numerator = a
```

```
        self.__denominator = b
        self.__reduce()

    # Сокращение дроби.
    def __reduce(self):
        # Функция для нахождения наибольшего общего делителя
        def gcd(a, b):
            if a == 0:
                return b
            elif b == 0:
                return a
            elif a >= b:
                return gcd(a % b, b)
            else:
                return gcd(a, b % a)

        sign = 1
        if (self.__numerator > 0 and self.__denominator < 0) or \
           (self.__numerator < 0 and self.__denominator > 0):
            sign = -1

        a, b = abs(self.__numerator), abs(self.__denominator)
        c = gcd(a, b)

        self.__numerator = sign * (a // c)
        self.__denominator = b // c

    # Клонировать дробь.
    def __clone(self):
```

```
    return Rational(self.__numerator, self.__denominator)

@property
def numerator(self):
    return self.__numerator

@numerator.setter
def numerator(self, value):
    self.__numerator = int(value)
    self.__reduce()

@property
def denominator(self):
    return self.__denominator

@denominator.setter
def denominator(self, value):
    value = int(value)
    if value == 0:
        raise ValueError("Illegal value of the denominator")

    self.__denominator = value
    self.__reduce()

def __str__(self):
    return f"{self.__numerator} / {self.__denominator}"

def __repr__(self):
    return self.__str__()
```

```
def __float__(self):
    return self.__numerator / self.__denominator

# Привести дробь к логическому значению.
def __bool__(self):
    return self.__numerator != 0

def __iadd__(self, rhs):  # +=
    if isinstance(rhs, Rational):
        a = self.numerator * rhs.denominator + \
            self.denominator * rhs.numerator
        b = self.denominator * rhs.denominator

        self.__numerator, self.__denominator = a, b
        self.__reduce()
        return self
    else:
        raise ValueError("Illegal type of the argument")

def __add__(self, rhs):  # +
    return self.__clone().__iadd__(rhs)
```

3. Выполнили предложенные задания по варианту

Задание №1:

Используя замыкания функций, объявите внутреннюю функцию, которая принимает в качестве аргумента коллекцию (список или кортеж) и возвращает или минимальное значение, или максимальное, в зависимости от значения параметра type внешней функции. Если type равен «max», то

возвращается максимальное значение, иначе – минимальное. По умолчанию type должно принимать значение «max». Вызовите внутреннюю функцию замыкания и отобразите на экране результат ее работы. Выполнить это задание, максимально задействуя перегрузки операторов.

Листинг кода z1.py:

```
class SmartNumber:
    def __init__(self, value):
        self.value = value

    def __gt__(self, other):
        print(f"  [Перегрузка >]: Сравниваем {self.value} > {other.value}")
        return self.value > other.value

    def __lt__(self, other):
        print(f"  [Перегрузка <]: Сравниваем {self.value} < {other.value}")
        return self.value < other.value

    def __eq__(self, other):
        print(f"  [Перегрузка ==]: Проверяем {self.value} == {other.value}")
        return self.value == other.value

    def __repr__(self):
        return str(self.value)


def create_finder(type="max"):

    def find(collection):
        if not collection:
```

```
            return None

        if type == "max":
            return max(collection)

        else:
            return min(collection)

    return find

numbers = [SmartNumber(5), SmartNumber(12), SmartNumber(3),
SmartNumber(8)]

max_finder = create_finder()
result_max = max_finder(numbers)
print(f"Результат max: {result_max}\n")

min_finder = create_finder(type="min")
result_min = min_finder(numbers)
print(f"Результат min: {result_min}")
```

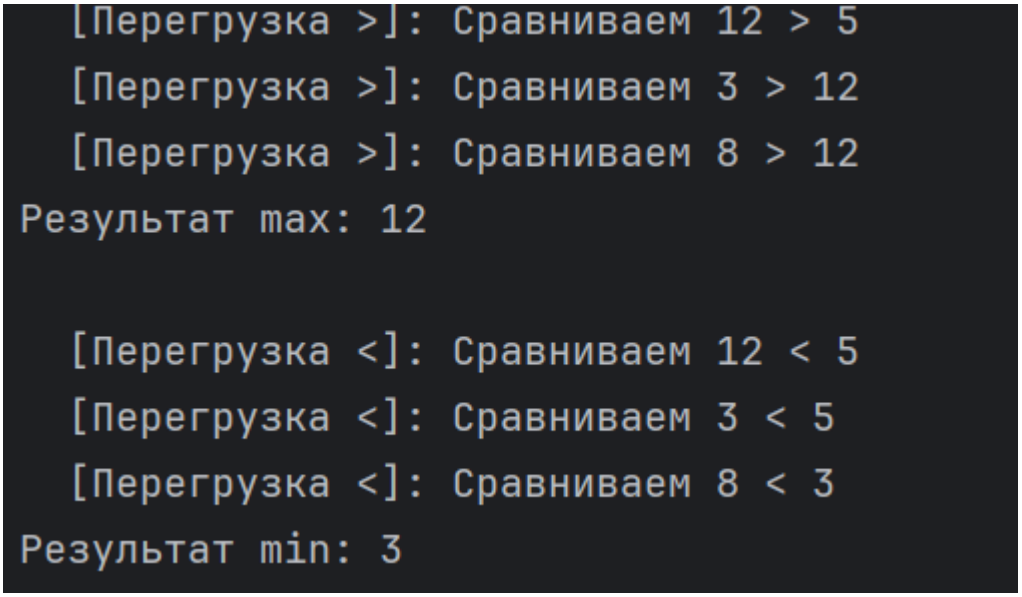


Рис. 1 — Выполнение программы

Задание №2:

Дополнительно к требуемым в заданиях операциям перегрузить операцию индексирования []. Максимально возможный размер списка задать константой. В отдельном поле size должно храниться максимальное для данного объекта количество элементов списка; реализовать метод size(), возвращающий установленную длину. Если количество элементов списка изменяется во время работы, определить в классе поле count. Первоначальные значения size и count устанавливаются конструктором.

В тех задачах, где возможно, реализовать конструктор инициализации строкой.

9. Карточка иностранного слова представляет собой словарю, содержащую иностранное слово и его перевод. Для моделирования электронного словаря иностранных слов реализовать класс Dictionary. Данный класс имеет поле-название словаря и содержит список словарей WordCard, представляющих собой карточки иностранного слова. Название словаря задается при создании нового словаря, но должна быть предоставлена возможность его изменения во время работы. Карточки добавляются в словарь и удаляются из него. Реализовать поиск

определенного слова как отдельный метод. Аргументом операции индексирования должно быть иностранное слово. В словаре не должно быть карточек-дублей. Реализовать операции объединения, пересечения и вычитания словарей. При реализации должен создаваться новый словарь, а исходные словари не должны изменяться. При объединении новый словарь должен содержать без повторений все слова, содержащиеся в обоих словарях-операндах. При пересечении новый словарь должен состоять только из тех слов, которые имеются в обоих словарях-операндах. При вычитании новый словарь должен содержать слова первого словаря-операнда, отсутствующие во втором.

Листинг кода z2.py:

```
MAX_SIZE = 100

class WordCard:
    def __init__(self, foreign_word, translation):
        self.foreign_word = foreign_word
        self.translation = translation

    def __eq__(self, other):
        if isinstance(other, WordCard):
            return self.foreign_word == other.foreign_word
        return False

    def __hash__(self):
        return hash(self.foreign_word)

    def __repr__(self):
        return f"WordCard('{self.foreign_word}', '{self.translation}')"
```

```
class Dictionary:
    def __init__(self, name="Новый словарь"):
        self.name = name
        self.size = MAX_SIZE
        self.count = 0
        self._cards = []

    @classmethod
    def from_string(cls, name, data_string):
        dictionary = cls(name)
        if data_string.strip():
            pairs = data_string.split(',')
            for pair in pairs:
                if ':' in pair:
                    word, trans = pair.split(':')
                    dictionary.add_card(WordCard(word.strip(), trans.strip()))
        return dictionary

    def change_name(self, new_name):
        self.name = new_name

    def add_card(self, card):
        if self.count >= self.size:
            raise OverflowError("Словарь заполнен (достигнут максимальный размер)")

        if card not in self._cards:
```

```
        self._cards.append(card)
        self.count += 1

    def remove_card(self, foreign_word):
        for card in self._cards:
            if card.foreign_word == foreign_word:
                self._cards.remove(card)
                self.count -= 1
                return True
        return False

    def search(self, foreign_word):
        for card in self._cards:
            if card.foreign_word == foreign_word:
                return card.translation
        return None

    def size_method(self):
        return self.size

    def __getitem__(self, foreign_word):
        translation = self.search(foreign_word)
        if translation is not None:
            return translation
        raise KeyError(f"Слово '{foreign_word}' не найдено в словаре")

    def __setitem__(self, foreign_word, translation):
        for card in self._cards:
            if card.foreign_word == foreign_word:
```

```
                card.translation = translation
                return
        self.add_card(WordCard(foreign_word, translation))

    def __or__(self, other):
        if not isinstance(other, Dictionary):
            return NotImplemented

        new_dict = Dictionary(f"{self.name} | {other.name}")
        for card in self._cards:
            new_dict.add_card(WordCard(card.foreign_word,
card.translation))
        for card in other._cards:
            new_dict.add_card(WordCard(card.foreign_word,
card.translation))
        return new_dict

    def __and__(self, other):
        if not isinstance(other, Dictionary):
            return NotImplemented

        new_dict = Dictionary(f"{self.name} & {other.name}")
        other_words = {card.foreign_word for card in other._cards}
        for card in self._cards:
            if card.foreign_word in other_words:
                new_dict.add_card(WordCard(card.foreign_word,
card.translation))
        return new_dict
```

```
        def __sub__(self, other):
            if not isinstance(other, Dictionary):
                return NotImplemented

            new_dict = Dictionary(f"{self.name} - {other.name}")
            other_words = {card.foreign_word for card in other._cards}
            for card in self._cards:
                if card.foreign_word not in other_words:
                    new_dict.add_card(WordCard(card.foreign_word,
card.translation))
            return new_dict

        def __str__(self):
            return f"Словарь '{self.name}' (карточек: {self.count}/{self.size})"


    if __name__ == "__main__":
        dict1 = Dictionary("Англо-русский")
        dict1.add_card(WordCard("apple", "яблоко"))
        dict1.add_card(WordCard("dog", "собака"))
        dict1.add_card(WordCard("cat", "кошка"))

        dict2 = Dictionary.from_string("Мой словарь", "dog:пес, bird:птица,
cat:кот")

        print(dict1)
        print(dict2)

        print(f"Поиск слова 'dog': {dict1.search('dog')}")
```

```
print(f"Индексация dict1['apple']: {dict1['apple']}")

dict_union = dict1 | dict2
print(dict_union)
print(f"Перевод 'dog' в объединенном: {dict_union['dog']}")

dict_intersection = dict1 & dict2
print(dict_intersection)

dict_subtraction = dict1 - dict2
print(dict_subtraction)

print(f"Максимальный размер (size): {dict1.size}")
print(f"Текущее количество (count): {dict1.count}")
print(f"Вызов метода size(): {dict1.size_method()}")
```

```
Словарь 'Англо-русский' (карточек: 3/100)
Словарь 'Мой словарь' (карточек: 3/100)
Поиск слова 'dog': собака
Индексация dict1['apple']: яблоко
Словарь 'Англо-русский | Мой словарь' (карточек: 4/100)
Перевод 'dog' в объединенном: собака
Словарь 'Англо-русский & Мой словарь' (карточек: 2/100)
Словарь 'Англо-русский - Мой словарь' (карточек: 1/100)
Максимальный размер (size): 100
Текущее количество (count): 3
Вызов метода size(): 100
```

Рис. 2 — Выполнение программы

3. Отправили все изменения в удалённый репозиторий на GitHub и удостоверились в отображении всех изменений.

Контрольные вопросы:

1. Какие средства существуют в Python для перегрузки операций? В Python перегрузка операций реализуется не через специальные ключевые слова (как, например, operator+ в C++), а с помощью магических (dunder) методов.

Магические методы — это специальные методы, имена которых начинаются и заканчиваются двумя подчеркиваниями (например, __add__, __str__). Они вызываются интерпретатором Python автоматически, когда объект участвует в определенной операции (сложение, сравнение, вывод на экран, индексация и т.д.).

2. Какие существуют методы для перегрузки арифметических операций и операций отношения в языке Python?

Арифметические операции:

__add__ / __radd__ — сложение (+)

__sub__ / __rsub__ — вычитание (-)

__mul__ / __rmul__ — умножение (*)

__truediv__ / __rtruediv__ — деление (/)

__floordiv__ / __rfloordiv__ — целочисленное деление (//)

__mod__ / __rmod__ — остаток от деления (%)

__pow__ / __rpow__ — возведение в степень (**)

__matmul__ — матричное умножение (@)

Операции отношения (сравнения):

__eq__ — равно (==)

__ne__ — не равно (!=)

__lt__ — меньше (<)

__le__ — меньше или равно (<=)

__gt__ — больше (>)

__ge__ — больше или равно (>=)

3. В каких случаях будут вызваны следующие методы: __add__, __iadd__ и __radd__? Приведите примеры.

__add__ вызывается при обычном сложении, когда объект нашего класса стоит слева от знака +.

Пример: obj + 5 или obj1 + obj2.

__iadd__ вызывается при использовании оператора присваивания со сложением +=. Этот метод позволяет изменять объект "на месте" (in-place), не создавая новый.

Пример: obj += 5. Если __iadd__ не определен, Python сделает то же самое, что и obj = obj + 5 (вызовет __add__ и перезапишет ссылку).

__radd__ (right add) вызывается, когда наш объект стоит справа от знака +, а левый операнд (например, стандартное число) не знает, как сложиться с нашим объектом.

Пример: 5 + obj. Python сначала попытается вызвать 5.__add__(obj), потерпит неудачу (число не знает, что делать с вашим классом) и тогда вызовет obj.__radd__(5).

4. Для каких целей предназначен метод __new__? Чем он отличается от метода __init__?

Для чего нужен __new__: Он предназначен для создания нового экземпляра класса (выделения памяти). Это статический метод, который вызывается до __init__ и возвращает созданный объект.

Чем отличается от __init__:

__new__ создает объект (конструктор).

__init__ инициализирует уже созданный объект (устанавливает начальные значения атрибутов).

На практике __new__ переопределяют редко (например, при наследовании от неизменяемых типов вроде int, str, tuple, или при реализации паттерна Singleton). В 99% случаев достаточно использовать __init__.

5. Чем отличаются методы __str__ и __repr__?

Оба метода возвращают строковое представление объекта, но для разных целей:

__str__ — используется для пользователя. Его цель — вернуть читаемое, красивое строковое представление. Вызывается функциями print() и str().

Пример: Для даты __str__ вернет "2023-10-05".

__repr__ — используется для разработчика (отладка). Его цель — вернуть однозначное представление, по которому в идеале можно воссоздать объект. Вызывается функцией repr() и используется в интерактивной консоли.

Пример: Для даты __repr__ вернет "datetime.date(2023, 10, 5)".

Связь: Если метод __str__ не определен в классе, Python автоматически использует __repr__ при вызове print().

Вывод: приобрели навыки по перегрузке операторов при написании программ с помощью языка программирования Python версии 3.x